

02; Random Graph Models

Networks I: Concepts and Algorithms | BSE Data Science

Notes based on Milán (2026), Newman (2013), Easley & Kleinberg (2010)

2026-03-20

Table of contents

1	The Case for Random Graph Models	2
1.1	From Description to Inference	2
1.2	A Roadmap of Models (and Their Failures)	3
2	Erdős–Rényi: The Simplest Random Graph	3
2.1	Model Definition	3
2.2	Expected Properties	4
2.3	Degree Distribution: From Binomial to Poisson	4
2.4	Clustering: A Critical Failure	5
2.5	Phase Transitions and the Giant Component	8
2.5.1	The Phase Transition Phenomenon	8
2.5.2	Deriving the Giant Component Equation	8
2.5.3	Full Connectivity	9
2.5.4	Contagion on ER Graphs (Application)	11
3	The Configuration Model: Arbitrary Degree Sequences	12
3.1	Why We Need More Than ER	12
3.2	Key Properties	12
3.3	The Friendship Paradox	13
4	The Watts–Strogatz Small World Model	16
4.1	The Core Tension	16
4.2	The Model	17
4.3	Limitations of Watts–Strogatz	19
5	Power Laws and Preferential Attachment	19
5.1	The Empirical Pattern	19

5.2	The Barabási–Albert Model: Preferential Attachment	20
5.2.1	Mean-Field Analysis	20
5.2.2	Deriving the Power Law	21
5.2.3	Maximum Hub Size	21
5.3	The Jackson–Rogers Hybrid Model	24
6	Stochastic Block Models: Networks with Community Structure	25
6.1	From Descriptive to Generative	25
6.2	Model Definition	25
6.3	The Planted Partition Model	25
6.4	MLE Inference for SBM	26
6.5	Degree-Corrected SBM (DC-SBM)	28
7	Summary and What’s Next	28
7.1	Model Comparison	28
7.2	What We Cannot Yet Answer	29
7.3	References	29

1 The Case for Random Graph Models

1.1 From Description to Inference

In Lecture 1 we learned to *describe* networks: compute degree distributions, path lengths, clustering coefficients, centralities. These are useful summaries, but they leave open the most important question: **is what we observe surprising?**

Suppose we find that the American75 Facebook network has transitivity $C = 0.15$. Is that high or low? We cannot answer without a **baseline**; a reference distribution of what C looks like in “generic” networks with the same basic parameters (n nodes, $\langle k \rangle$ average degree).

This is the role of **random graph models**: they provide principled null distributions against which to measure real-world structure.

i Three uses of random graph models

1. **Null model testing**: is the clustering/modularity/diameter of my network significantly different from what a random graph would produce?
2. **Generative understanding**: what simple mechanisms give rise to the patterns we observe (power laws, small worlds, community structure)?
3. **Analytical tractability**: random graphs admit exact or asymptotic calculations that real networks do not.

1.2 A Roadmap of Models (and Their Failures)

We will introduce models in order of increasing realism, and crucially, each model's *failure* motivates the next:

Model	What it gets right	What it misses
Erdős–Rényi $G(n, p)$	Degree distribution tractable, phase transition	Clustering $\rightarrow 0$, Poisson tails
Configuration model	Arbitrary degree sequence	Clustering $\rightarrow 0$, no community structure
Watts–Strogatz	High clustering AND short paths	Degree distribution still Poisson-like
Barabási–Albert	Power-law degree distribution	No clustering, no communities
Jackson–Rogers hybrid	Power law + positive clustering	Computationally heavier
Stochastic Block Model	Community structure	Assumes homogeneous within-community degree
DC-SBM	Community structure + degree heterogeneity	More parameters

2 Erdős–Rényi: The Simplest Random Graph

2.1 Model Definition

The **Erdős–Rényi model** $G(n, p)$ (Gilbert 1959; Erdős & Rényi 1959–60) is the simplest possible random graph: each of the $\binom{n}{2}$ potential edges exists independently with probability p .

The adjacency matrix entries are i.i.d. Bernoulli:

$$A_{ij} = A_{ji} \sim \text{Bernoulli}(p) \quad \forall i < j$$

The probability of drawing a particular graph with m edges is:

$$\Pr(G) = p^m (1 - p)^{\binom{n}{2} - m}$$

The $\mathbf{G}(\mathbf{n}, \mathbf{m})$ variant fixes the *number* of edges at exactly m , choosing uniformly among all graphs with that edge count. As we will see (PS1 Q3), the two models are equivalent for large n when $m = \langle m \rangle$.

2.2 Expected Properties

Expected degree: each node can link to $n - 1$ others, each with probability p :

$$\langle k \rangle = (n - 1)p$$

Expected edges: $\langle m \rangle = \binom{n}{2}p = \frac{n(n-1)}{2}p$

To study “interesting” limiting behaviour we want $\langle k \rangle = c$ fixed as $n \rightarrow \infty$. This requires $p = c/(n - 1) \rightarrow 0$, so the graph becomes *sparser* as it grows.

2.3 Degree Distribution: From Binomial to Poisson

Each node independently connects to each of the other $n - 1$ nodes with probability p , so degree follows a Binomial:

$$\Pr(k) = \binom{n-1}{k} p^k (1-p)^{n-1-k}$$

For $p = c/(n - 1)$ and $n \rightarrow \infty$, this converges to a **Poisson** distribution:

$$\Pr(k) \approx \frac{c^k}{k!} e^{-c}$$

The derivation uses $\binom{n-1}{k} p^k \approx c^k/k!$ and $(1-p)^{n-1-k} \approx e^{-c}$ for large n . Both mean and variance of this Poisson are equal to c .

Why this matters (and fails): the Poisson distribution has *exponentially* decaying tails. The probability of a node having degree $10c$ drops like $(10c)^{10c} e^{-10c}/(10c)!$, which is vanishingly small. Real networks have **fat tails**: hubs with degree far exceeding the mean are common, not exponentially rare. We will return to this when discussing power laws.

2.4 Clustering: A Critical Failure

The global clustering coefficient in $G(n, p)$ is simply:

$$C = \frac{\text{triangles} \times 3}{\text{connected triples}} \approx \frac{\binom{n}{3} p^3}{\binom{n}{3} p^2} = p = \frac{c}{n-1}$$

As $n \rightarrow \infty$ with c fixed: $C \rightarrow 0$. ER graphs are **locally tree-like**: there are essentially no triangles, and no cycles of any finite length, in the large- n limit.

This is disqualifying as a model for social networks. Real friendship networks have C in the range 0.1–0.5, while ER predicts $C \sim 10^{-3}$ for the same parameters. The “friend of my friend is also my friend” structure of human social life is absent by construction in ER.

```
set.seed(42)
c_target <- 5.0

## (A) Degree distribution: simulated vs Poisson theory
par(mfrow = c(1, 3), mar = c(4, 4, 4, 1))

# Overlapping histograms for n = 100, 500, 2000
all_n <- c(100, 500, 2000)
all_cols <- c("#aec7e8", "#1f77b4", "#084594")
k_range <- 0:19

# Build density matrices for plotting
plot(NULL, xlim = c(0, 19), ylim = c(0, 0.25),
      xlab = "Degree k", ylab = "P(k)",
      main = "Degree → Poisson as n → ∞")
for (i in seq_along(all_n)) {
  n <- all_n[i]
  p <- c_target / (n - 1)
  g_t <- erdos.renyi.game(n = n, p = p, type = "gnp")
  d <- degree(g_t)
  # Use hist with add=TRUE trick via density
  h <- hist(d, breaks = -1:20 + 0.5, plot = FALSE)
  rect(h$breaks[-length(h$breaks)], 0, h$breaks[-1], h$density,
      col = adjustcolor(all_cols[i], alpha.f = 0.4), border = NA)
}
lines(k_range, dpois(k_range, lambda = c_target),
      col = "red", lwd = 2, type = "b", pch = 19, cex = 0.6)
legend("topright", legend = c("n=100", "n=500", "n=2000",
                              paste0("Poisson(c=", c_target, ")")),
```

```

    fill    = c(adjustcolor(all_cols, 0.5), NA),
    lty     = c(NA, NA, NA, 1), col = c(NA, NA, NA, "red"),
    border  = c(all_cols, NA), bty = "n", cex = 0.8)

## (B) Clustering vs n (log-y scale)
ns      <- c(50, 100, 200, 500, 1000, 2000)
c_vals  <- c(3, 5, 10)
col_b   <- c("#4C72B0", "#DD8452", "#55A868")

plot(NULL, xlim = range(ns), ylim = c(1e-4, 0.4),
     log = "y",
     xlab = "n", ylab = "C(A) [log scale]",
     main = "Clustering C → 0 as n → ∞\n(dashed = theory p = c/(n-1))")
for (j in seq_along(c_vals)) {
  c_v      <- c_vals[j]
  obs_C    <- numeric(length(ns))
  er_pred  <- numeric(length(ns))
  for (i in seq_along(ns)) {
    n <- ns[i]
    p <- c_v / (n - 1)
    g_t    <- erdos.renyi.game(n = n, p = p, type = "gnp")
    obs_C[i] <- transitivity(g_t, type = "global")
    er_pred[i] <- p
  }
  lines(ns, obs_C, col = col_b[j], lwd = 2, type = "b", pch = 19, cex = 0.5)
  lines(ns, er_pred, col = col_b[j], lwd = 1.5, lty = 2)
}
legend("topright", legend = paste0("c=", c_vals, " (sim)"),
      col = col_b, lwd = 2, bty = "n", cex = 0.8)

## (C) Real Facebook vs ER null
mat <- readMat("American75.mat")
A_sp <- mat$A
A_t <- as(A_sp, "TsparseMatrix")
edges_fb <- cbind(A_t@i + 1L, A_t@j + 1L)
g_fb <- graph_from_edgelist(edges_fb, directed = FALSE)
g_fb <- simplify(g_fb)

n_fb <- vcount(g_fb)
c_fb <- mean(degree(g_fb))
p_fb <- c_fb / (n_fb - 1)
g_er_fb <- erdos.renyi.game(n = n_fb, p = p_fb, type = "gnp")

```

```

C_real <- transitivity(g_fb, type = "global")
C_er <- transitivity(g_er_fb, type = "global")
labels <- c("American75\n(real)", "G(n,p)\n(ER null)")
C_vals_bar <- c(C_real, C_er)
bar_cols <- c("#C44E52", "#4C72B0")

bp <- barplot(C_vals_bar, names.arg = labels, col = bar_cols, border = NA,
             ylab = "Global clustering C(A)",
             main = sprintf("Real vs ER\n(same n=%d, <k>=%.1f)", n_fb, c_fb),
             ylim = c(0, max(C_vals_bar) * 1.2))
text(bp, C_vals_bar + 0.003, sprintf("%.4f", C_vals_bar),
     cex = 0.9, adj = 0.5)

```

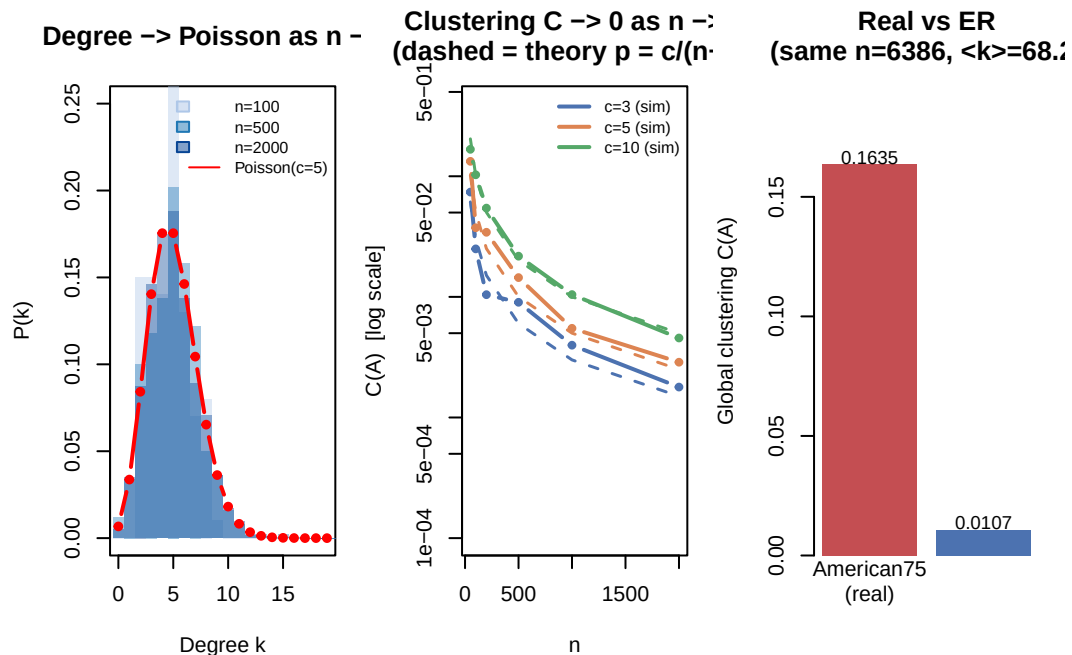


Figure 1: Degree distribution of $G(n,p)$ approaches Poisson as n grows. Clustering vanishes.

```
cat(sprintf("American75: C = %.4f\n", C_real))
```

American75: C = 0.1635

```
cat(sprintf("ER null: C = %.5f\n", C_er))
```

ER null: C = 0.01071

```
cat(sprintf("Ratio (real/ER): %.0fx higher\n", C_real / C_er))
```

Ratio (real/ER): 15x higher

2.5 Phase Transitions and the Giant Component

2.5.1 The Phase Transition Phenomenon

Perhaps the most beautiful result in random graph theory is the sudden emergence of a **giant component**; a connected component containing a finite fraction of all nodes; as the average degree c crosses 1.

! Erdős–Rényi Phase Transition

At $c = \langle k \rangle = 1$, there is a sharp phase transition:

- **Subcritical** ($c < 1$): all components have size $O(\log n)$. The largest component as a fraction of n vanishes.
- **Supercritical** ($c > 1$): a **giant component** of size $\sim Sn$ exists, where $S > 0$ solves $S = 1 - e^{-cS}$.
- **Critical** ($c = 1$): largest component is $O(n^{2/3})$; diverging, but sub-linear in n .

2.5.2 Deriving the Giant Component Equation

Let u be the probability that a randomly chosen node is *not* in the giant component. Node i is not in the GC if, for every other node j , *either* there is no edge (i, j) (probability $1 - p$) *or* there is an edge but j is not in the GC (probability pu). Since edges are independent:

$$u = (1 - p + pu)^{n-1}$$

Substituting $p = c/(n-1)$ and taking logs:

$$\ln u = (n-1) \ln \left(1 - \frac{c(1-u)}{n-1} \right) \approx -(n-1) \cdot \frac{c(1-u)}{n-1} = -c(1-u)$$

Letting $S = 1 - u$ (the fraction *in* the GC):

$$S = 1 - e^{-cS}$$

This transcendental equation has no closed-form solution, but we can solve it graphically or numerically. For $c \leq 1$: the only solution is $S = 0$. For $c > 1$: a unique positive solution $S \in (0, 1)$ exists.

2.5.3 Full Connectivity

Once the giant component absorbs almost all nodes, we can ask: when does the *entire* network become connected (no isolated nodes)?

An isolated node survives if all $n - 1$ potential neighbours are absent. The expected number of isolated nodes is:

$$I_n = n(1 - p)^{n-1} \approx ne^{-np} \approx ne^{-c}$$

For the last isolated node to disappear ($I_n = 1$), we need $p \sim \ln(n)/n$, corresponding to $c = \langle k \rangle \sim \ln n$.

Summary of phase transitions:

p (order)	Event
$1/n^2$	First edge appears
$1/n^{3/2}$	First triple (path of length 2)
$1/n$	Giant component emerges ($c = 1$)
$\ln n/n$	Graph becomes fully connected

```
## Theoretical S via bisection
gc_theory <- function(c) {
  if (c <= 1) return(0.0)
  f <- function(S) S - (1 - exp(-c * S))
  uniroot(f, c(1e-9, 1 - 1e-9))$root
}

set.seed(42)
n <- 1000
c_range <- seq(0, 4, length.out = 80)

gc_frac_sim <- sapply(c_range, function(c) {
  p <- c / (n - 1)
```

```

    g_tmp <- erdos.renyi.game(n = n, p = p, type = "gnp")
    max(components(g_tmp)$csize) / n
  })

S_theory <- sapply(c_range, gc_theory)

par(mfrow = c(1, 2), mar = c(4, 4, 4, 1))

## (A) Simulated vs theory
plot(c_range, gc_frac_sim,
     pch = 19, cex = 0.5, col = adjustcolor("#4C72B0", 0.6),
     xlab = expression(paste("Average degree ", c == langle*k*rangle)),
     ylab = "Giant component size / n",
     main = expression(paste("Phase Transition in ", G(n, p))),
     ylim = c(-0.02, 1.02))
lines(c_range, S_theory, col = "red", lwd = 2)
abline(v = 1, lty = 2, lwd = 1.5, col = "black")
legend("topleft",
      legend = c(paste0("Simulated (n=", n, ")"),
                  expression(paste("Theory: ", S == 1 - e^{-cS})),
                  "Critical point c=1"),
      col = c(adjustcolor("#4C72B0", 0.6), "red", "black"),
      pch = c(19, NA, NA), lty = c(NA, 1, 2), lwd = c(NA, 2, 1.5),
      bty = "n", cex = 0.85)

## (B) Fixed-point geometry
S_vals <- seq(0, 1, length.out = 200)
c_fp <- c(0.5, 1.0, 2.0, 3.0)
fp_cols <- c("#aec7e8", "#DD8452", "#C44E52", "#8B0000")
fp_labs <- c("c=0.5 (sub)", "c=1.0 (critical)", "c=2.0 (super)", "c=3.0 (super)")

plot(NULL, xlim = c(0, 1), ylim = c(0, 1),
     xlab = "S", ylab = expression(1 - e^{-cS}),
     main = expression(paste("Fixed-point: ", S == 1 - e^{-cS},
                              "\n(intersection = GC fraction)")))
for (j in seq_along(c_fp)) {
  lines(S_vals, 1 - exp(-c_fp[j] * S_vals),
        col = fp_cols[j], lwd = 1.8)
}
abline(0, 1, lty = 2, lwd = 1.5)
legend("topleft", legend = c(fp_labs, "S (identity)"),
      col = c(fp_cols, "black"), lwd = 2,

```

```
lty = c(rep(1, 4), 2), bty = "n", cex = 0.85)
```

Phase Transition in $G(n, p)$: $S = 1 - e^{-cS}$ (intersection =

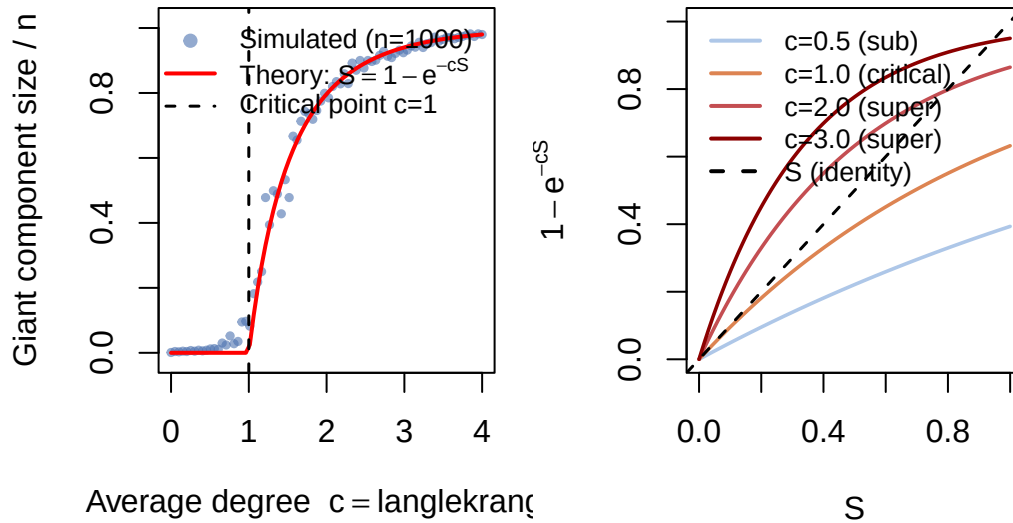


Figure 2: Phase transition in $G(n, p)$. The giant component jumps from 0 to $O(n)$ at $c=1$.

```
## Print GC fractions at key c values
for (c_v in c(0.5, 0.9, 1.0, 1.5, 2.0, 3.0)) {
  S <- gc_theory(c_v)
  diam_log <- log(1000) / log(max(c_v, 1.001))
  cat(sprintf("c = %.1f: theory S = %.4f, diameter ln(n)/ln(c) = %.1f\n",
              c_v, S, diam_log))
}
```

```
c = 0.5: theory S = 0.0000, diameter ln(n)/ln(c) = 6911.2
c = 0.9: theory S = 0.0000, diameter ln(n)/ln(c) = 6911.2
c = 1.0: theory S = 0.0000, diameter ln(n)/ln(c) = 6911.2
c = 1.5: theory S = 0.5828, diameter ln(n)/ln(c) = 17.0
c = 2.0: theory S = 0.7968, diameter ln(n)/ln(c) = 10.0
c = 3.0: theory S = 0.9405, diameter ln(n)/ln(c) = 6.3
```

2.5.4 Contagion on ER Graphs (Application)

The phase transition directly models **epidemic thresholds**. Consider an SI (susceptible-infected) model on $G(n, p)$:

- A fraction π of nodes are vaccinated (immune)
- Effective graph: $G((1 - \pi)n, p)$; remove immune nodes
- The disease spreads iff this effective graph has a giant component

The epidemic threshold is when the effective average degree exceeds 1: $c_{\text{eff}} = (1 - \pi)c > 1$, i.e., $\pi < 1 - 1/c$.

This is the **herd immunity threshold** in network terms. For the network to be protected, we must vaccinate at least fraction $1 - 1/c$ of nodes.

3 The Configuration Model: Arbitrary Degree Sequences

3.1 Why We Need More Than ER

Erdős–Rényi produces Poisson degree distributions. Real networks have heavy-tailed degree distributions; power laws, log-normals. The **configuration model** decouples the degree sequence from the wiring pattern: we can input *any* degree sequence $\{k_1, k_2, \dots, k_n\}$ and get a “random” graph that has those exact degrees.

The construction (Bender & Canfield 1978):

1. Give node i exactly k_i **stubs** (half-edges)
2. Choose a uniformly random perfect matching of all $\sum_i k_i$ stubs
3. Each matched pair of stubs becomes an edge

This produces multi-edges and self-loops with probability $O(1/n)$, which vanishes in large networks. For our purposes, we treat the resulting graph as simple.

3.2 Key Properties

Linking probability between nodes i and j :

$$p_{ij} = \frac{k_i k_j}{\sum_l k_l} = \frac{k_i k_j}{2m}$$

This is a *heterogeneous* ER model: high-degree nodes are more likely to connect to each other; and to everyone else.

Expected number of common neighbours between i and j :

$$N_{ij} = \sum_l p_{il} p_{lj} = \frac{k_i k_j}{2m} \cdot \frac{\langle k^2 \rangle - \langle k \rangle}{\langle k \rangle}$$

Clustering still vanishes: $C = O(1/n)$ for fixed $\langle k \rangle$, just as in ER. The configuration model fixes the *degree sequence* but still wires edges randomly; no triangle-formation mechanism exists.

3.3 The Friendship Paradox

One of the most surprising properties of configuration models:

! Friendship Paradox (Feld 1991)

On average, your friends have more friends than you do.

Formally, the expected degree of a *uniformly chosen neighbour* is:

$$\langle k_{\text{nbr}} \rangle = \frac{\langle k^2 \rangle}{\langle k \rangle} = \langle k \rangle + \frac{\text{Var}(k)}{\langle k \rangle} \geq \langle k \rangle$$

with equality only when all degrees are equal.

Why: when you follow a random edge, the probability of landing at node j is proportional to k_j (high-degree nodes contribute more stubs, so are more likely to be at the other end of a randomly chosen edge). This *size-biased sampling* over-represents hubs.

Economic consequences (Jackson 2019, JPE): the friendship paradox creates *systematic misperceptions*. If your friends are on average richer, healthier, or better-connected than you, you may mis-calibrate social norms and make suboptimal choices. Jackson shows this can cascade into information avalanches: even though only 1/3 of the population genuinely prefers choice A, biased sampling through social networks can make it look like the unanimous favourite.

Giant component in the configuration model: emerges when $\langle k^2 \rangle - 2\langle k \rangle > 0$. For a Poisson distribution ($\langle k^2 \rangle = \langle k \rangle^2 + \langle k \rangle$), this reduces to $\langle k \rangle > 1$; recovering the ER result. For heavy-tailed distributions, the condition is weaker: networks with power-law degree distributions essentially always have a giant component.

```
## Load American75 (re-use g_fb from earlier chunk, or reload)
mat      <- readMat("American75.mat")
A_sp     <- mat$A
A_t      <- as(A_sp, "TsparseMatrix")
edges_fb <- cbind(A_t@i + 1L, A_t@j + 1L)
```

```

g_real    <- graph_from_edgelist(edges_fb, directed = FALSE)
g_real    <- simplify(g_real)

degrees   <- degree(g_real)
n_nodes   <- vcount(g_real)

# Mean degree of each node's neighbourhood
nbr_mean_k <- sapply(seq_len(n_nodes), function(v) {
  nbrs <- neighbors(g_real, v)
  if (length(nbrs) == 0) return(0.0)
  mean(degrees[nbrs])
})

mean_k     <- mean(degrees)
mean_k2    <- mean(degrees^2)
theory_nbr <- mean_k2 / mean_k

mask <- degrees >= 2

par(mfrow = c(1, 2), mar = c(4, 4, 4, 1))

## (A) Scatter: own degree vs mean neighbour degree
plot(degrees[mask], nbr_mean_k[mask],
     pch = 19, cex = 0.4, col = adjustcolor("#4C72B0", 0.3),
     xlab = expression(paste("Own degree  ", k[i])),
     ylab = "Mean neighbour degree",
     main = "Friendship Paradox\n(points above red line = paradox)")
abline(0, 1, col = "red", lty = 2, lwd = 1.5)
legend("topleft", legend = "Equal line", col = "red", lty = 2, bty = "n")

## (B) Distribution comparison
max_deg <- max(degrees)
h_own <- hist(degrees,
             breaks = seq(-0.5, max_deg + 0.5, 1),
             plot = FALSE)
h_nbr <- hist(nbr_mean_k[mask],
             breaks = seq(0, max(nbr_mean_k[mask]) + 5, 5),
             plot = FALSE)

plot(h_own$mids, h_own$density, type = "s", col = "#4C72B0", lwd = 2,
     xlab = "Degree", ylab = "Density",
     main = "Degree distributions",
     xlim = c(0, quantile(nbr_mean_k[mask], 0.99)),
     ylim = c(0, max(h_own$density) * 1.1))

```

```

lines(h_nbr$mids, h_nbr$density, type = "s", col = "#C44E52", lwd = 2)
abline(v = mean_k, col = "#4C72B0", lty = 2, lwd = 2)
abline(v = mean(nbr_mean_k[mask]), col = "#C44E52", lty = 2, lwd = 2)
abline(v = theory_nbr, col = "black", lty = 3, lwd = 2)
legend("topright",
      legend = c(sprintf("Own degree (mean=%.1f)", mean_k),
                  sprintf("Nbr degree (mean=%.1f)", mean(nbr_mean_k[mask])),
                  sprintf("Theory <k²>/<k> = %.1f", theory_nbr)),
      col = c("#4C72B0", "#C44E52", "black"),
      lwd = 2, lty = c(1, 1, 3), bty = "n", cex = 0.85)

```

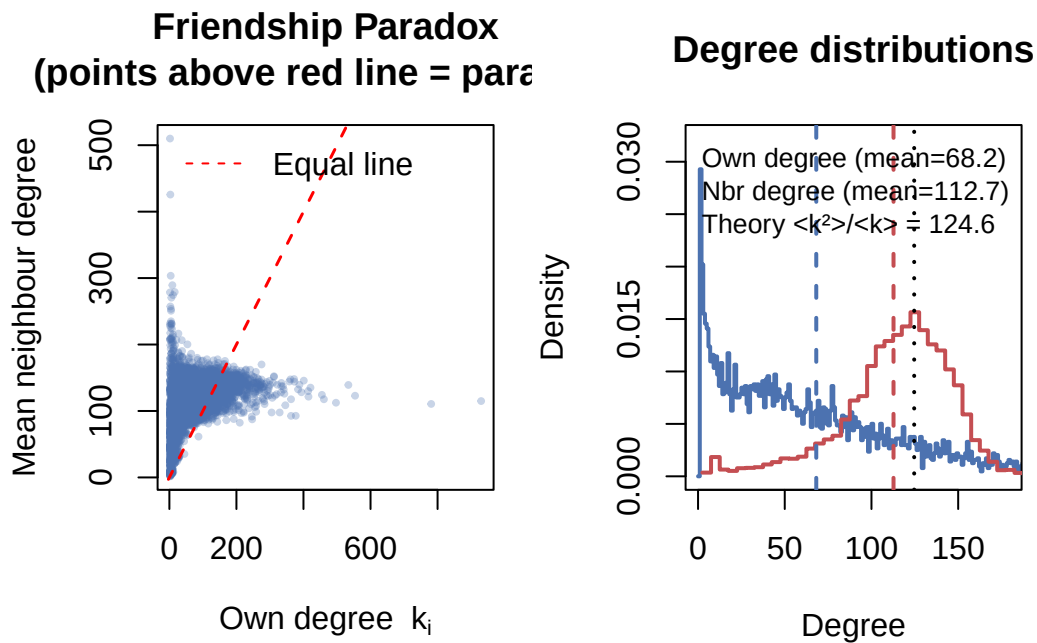


Figure 3: Friendship paradox: your neighbours' average degree exceeds your own.

```

cat(sprintf("Own mean degree:          <k>    = %.2f\n", mean_k))

```

Own mean degree: <k> = 68.17

```

cat(sprintf("Mean neighbour's degree:      = %.2f\n", mean(nbr_mean_k[mask])))

```

Mean neighbour's degree: = 112.72

```
cat(sprintf("Theory  $\langle k^2 \rangle / \langle k \rangle$ :                = %.2f\n", theory_nbr))
```

Theory $\langle k^2 \rangle / \langle k \rangle$: = 124.63

```
cat(sprintf("Var(k)/<k> (excess):                = %.2f\n",  
            mean_k2 / mean_k - mean_k))
```

Var(k)/<k> (excess): = 56.46

```
cat(sprintf("\nFriendship paradox ratio: %.2fx\n",  
            mean(nbr_mean_k[mask]) / mean_k))
```

Friendship paradox ratio: 1.65x

4 The Watts–Strogatz Small World Model

4.1 The Core Tension

We have two stylised facts about real social networks:

1. **High clustering:** people in the same community know each other ($C \sim 0.1\text{--}0.5$).
2. **Short average path lengths:** “six degrees of separation” suggests $\langle \ell \rangle = O(\log n)$.

These seem contradictory. High clustering requires lots of local triangles; which means most of your edges connect to people already in your cluster. Short paths require long-range connections that skip across the network. How can both hold simultaneously?

Erdős–Rényi achieves short paths (random wiring is great for connectivity) but kills clustering. Can we do better?

4.2 The Model

Watts & Strogatz (1998) propose a one-parameter interpolation:

1. **Start** with a regular ring lattice: n nodes arranged in a circle, each connected to its k nearest neighbours (so each node has degree k). This has high clustering and $O(n/k)$ diameter.
2. **Rewire** each edge independently with probability β : one endpoint is kept, the other is replaced with a uniformly random node.

At $\beta = 0$: original lattice; high C , large $\langle \ell \rangle$. At $\beta = 1$: Erdős-Rényi; low C , small $\langle \ell \rangle$.

The key insight: a *small number* of rewirings (small $\beta > 0$) dramatically collapses path lengths while leaving clustering nearly intact. This is because one “shortcut” across the network can reduce distances between two entire clusters from $O(n)$ to $O(1)$.

Clustering of the lattice (exact formula for even k):

$$C_{\text{lattice}} = \frac{3(k-2)}{4(k-1)} \approx \frac{3}{4} \quad (k \gg 1)$$

```
set.seed(42)
n_ws <- 200
k_ws <- 6 # each node connects to k_ws nearest neighbours

# Baseline lattice (beta = 0)
g0 <- sample_smallworld(dim = 1, size = n_ws, nei = k_ws / 2, p = 0)
C0 <- transitivity(g0, type = "global")
L0 <- mean_distance(g0, directed = FALSE)

betas <- 10^seq(-3, 0, length.out = 30)
C_norm <- numeric(length(betas))
L_norm <- numeric(length(betas))

for (i in seq_along(betas)) {
  g_ws <- sample_smallworld(dim = 1, size = n_ws, nei = k_ws / 2, p = betas[i])
  C_norm[i] <- transitivity(g_ws, type = "global") / C0
  L_norm[i] <- mean_distance(g_ws, directed = FALSE) / L0
}

par(mar = c(4, 4, 4, 1))
plot(betas, C_norm,
     log = "x", type = "b", pch = 19, cex = 0.6,
     col = "#4C72B0", lwd = 2,
```

```

xlab = expression(paste("Rewiring probability  ", beta)),
ylab = "Normalised value",
main = sprintf("Watts-Strogatz Small World\n(n=%d, k=%d)", n_ws, k_ws),
ylim = c(0, 1.05))
lines(betas, L_norm, type = "b", pch = 15, cex = 0.6,
      col = "#C44E52", lwd = 2)
# Shade small-world regime
rect(1e-2, 0, 0.1, 1.05,
     col = adjustcolor("green", alpha.f = 0.12), border = NA)
legend("left",
      legend = c(expression(C(beta)/C(0)~~"(clustering)"),
                  expression(langle*l*range(beta)/langle*l*range(0)~~"(avg path length)"),
                  "Small-world regime"),
      col      = c("#4C72B0", "#C44E52", adjustcolor("green", 0.3)),
      lwd      = c(2, 2, 8), pch = c(19, 15, NA), bty = "n", cex = 0.85)
grid()

```

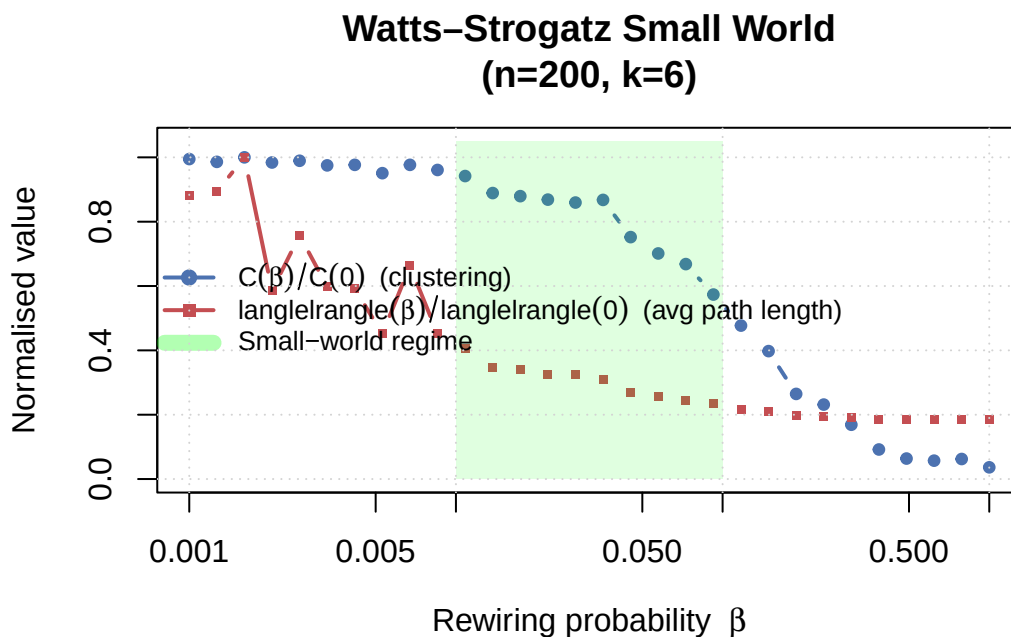


Figure 4: Watts–Strogatz: clustering stays high while diameter collapses for small .

```

cat(sprintf("At  = 0 (lattice): C = %.3f  L = %.2f\n", C0, L0))

```

```

At  = 0 (lattice): C = 0.600  L = 17.09

```

```
idx_half_L <- which(L_norm < 0.5)[1]
cat(sprintf("At    %.4f: L drops to 50%% of L0, C still at %.0f%% of C0\n",
           betas[idx_half_L], 100 * C_norm[idx_half_L]))
```

At 0.0053: L drops to 50% of L0, C still at 95% of C0

```
g_rnd <- sample_gnm(n = n_ws,
                   m = n_ws * k_ws / 2)
cat(sprintf("At    = 1 (ER approx): C = %.3f  L = %.2f\n",
           transitivity(g_rnd, type = "global"),
           mean_distance(g_rnd, directed = FALSE)))
```

At = 1 (ER approx): C = 0.036 L = 3.14

4.3 Limitations of Watts–Strogatz

While WS solves the clustering–diameter tension, it still produces a Poisson-like degree distribution (the rewiring changes *some* degrees slightly but does not create hubs). Real networks have heavy tails. This failure motivates the next model.

5 Power Laws and Preferential Attachment

5.1 The Empirical Pattern

Across an enormous range of systems; citations, hyperlinks, phone calls, metabolic networks; the degree distribution follows a **power law**:

$$p(k) \propto k^{-\gamma}$$

On a log-log plot, this appears as a straight line with slope $-\gamma$. Empirical exponents typically fall in the range $\gamma \in (2, 3)$.

The power law is *qualitatively* different from the Poisson: the standard deviation scales with the mean ($\sigma \sim \langle k \rangle$ for $\gamma \approx 3$, and even $\sigma = \infty$ for $\gamma \leq 3$). There is no “typical” degree; the distribution is **scale-free**.

i Why “scale-free”?

A function $f(x) = x^{-\gamma}$ satisfies $f(cx) = c^{-\gamma}f(x)$: scaling the argument by a constant changes the function by a constant factor, regardless of the starting value. There is no characteristic scale.

For the Poisson with mean c : the probability of degree $5c$ vs c drops by a factor e^{-4c} , which depends strongly on c . The Poisson has a scale (namely, c). The power law does not.

5.2 The Barabási–Albert Model: Preferential Attachment

Barabási & Albert (1999) proposed a simple growing network mechanism that generates power-law degree distributions:

1. Start with m_0 nodes.
2. At each timestep t , add one new node with m edges.
3. Each new edge connects to existing node i with probability **proportional to** $k_i(t)$ (“rich get richer”):

$$\Pr(\text{new edge} \rightarrow i) = \frac{k_i(t)}{\sum_j k_j(t)} = \frac{k_i(t)}{2t}$$

5.2.1 Mean-Field Analysis

Track the expected degree of node i as a continuous-time ODE. Node i gains one edge per timestep with probability $k_i/(2t)$:

$$\frac{dk_i}{dt} = m \cdot \frac{k_i(t)}{2t}$$

This is a separable ODE. Separating variables: $dk_i/k_i = m dt/(2t)$, integrating:

$$k_i(t) = m \left(\frac{t}{i} \right)^{1/2}$$

where i is the time node i was born (boundary condition $k_i(i) = m$). **Older nodes accumulate more links**; this is the *first-mover advantage* demonstrated empirically in citation networks (Newman’s papers, PS1 Q4).

5.2.2 Deriving the Power Law

From $k_i(t) = m\sqrt{t/i}$, the fraction of nodes with degree $\leq k$ at time t is the fraction of nodes born after time $t \cdot (m/k)^2$:

$$F_t(k) = \Pr(k_i(t) \leq k) = \Pr\left(i \geq \frac{m^2 t}{k^2}\right) = 1 - \frac{m^2 t / k^2}{t} = 1 - \frac{m^2}{k^2}$$

The density:

$$f(k) = \frac{dF}{dk} = \frac{2m^2}{k^3} \propto k^{-3}$$

The Barabási–Albert model always produces $\gamma = 3$, regardless of m . This is a specific prediction. Empirically γ varies, which motivates hybrid models.

5.2.3 Maximum Hub Size

For a network of n nodes with power-law exponent γ , the maximum degree scales as:

$$k_{\max} \sim k_{\min} \cdot n^{1/(\gamma-1)}$$

For $\gamma = 3$: $k_{\max} \sim \sqrt{n}$. For the WWW with $n \sim 10^9$: $k_{\max} \sim 30,000$; consistent with observed super-hubs.

```
set.seed(42)
par(mfrow = c(1, 3), mar = c(4, 4, 4, 1))

## (A) Degree CCDF of BA model
g_ba    <- sample_pa(n = 5000, m = 3, directed = FALSE)
deg_ba  <- degree(g_ba)
ks_sorted <- sort(unique(deg_ba))
ccdf_ba  <- sapply(ks_sorted, function(k) mean(deg_ba >= k))

plot(ks_sorted, ccdf_ba,
     log = "xy", pch = 19, cex = 0.4,
     col = adjustcolor("#4C72B0", 0.7),
     xlab = "Degree k", ylab = expression(bar(F)(k) ~ "(CCDF)"),
     main = "BA Model: Power-Law CCDF")
# OLS fit on log-log scale for k in [5, 200]
mask_fit <- ks_sorted >= 5 & ks_sorted <= 200
```

```

lm_fit    <- lm(log(ccdf_ba[mask_fit]) ~ log(ks_sorted[mask_fit]))
slope_fit <- coef(lm_fit)[2]
k_line    <- ks_sorted[mask_fit]
lines(k_line, exp(coef(lm_fit)[1]) * k_line^slope_fit,
      col = "red", lwd = 2, lty = 2)
legend("bottomleft",
      legend = c("BA model",
                  sprintf("Fit slope = %.2f\n(theory: -2)", slope_fit)),
      col     = c(adjustcolor("#4C72B0", 0.7), "red"),
      pch     = c(19, NA), lty = c(NA, 2), lwd = c(NA, 2),
      bty     = "n", cex = 0.85)

## (B) First-mover advantage
n_ba_fm <- 2000
m_ba    <- 2
g_ba_fm <- sample_pa(n = n_ba_fm, m = m_ba, directed = FALSE)
degs_fm <- degree(g_ba_fm)
birth_times <- seq_len(n_ba_fm)

q25 <- quantile(birth_times, 0.25)
q75 <- quantile(birth_times, 0.75)
early <- degs_fm[birth_times <= q25]
late  <- degs_fm[birth_times >= q75]

max_d <- max(c(early, late))
h_e <- hist(early, breaks = seq(-0.5, max_d + 0.5, 1), plot = FALSE)
h_l <- hist(late,  breaks = seq(-0.5, max_d + 0.5, 1), plot = FALSE)

plot(h_e$mids, h_e$density, type = "s", col = "#C44E52", lwd = 2,
     xlab = "Degree k", ylab = "Density",
     main = "First-Mover Advantage\n(Earlier nodes → higher degree)",
     xlim = c(0, quantile(early, 0.99) * 1.5),
     ylim = c(0, max(c(h_e$density, h_l$density)) * 1.1))
lines(h_l$mids, h_l$density, type = "s", col = "#4C72B0", lwd = 2)
legend("topright",
     legend = c(sprintf("Early nodes (q1)\nmean k=%.1f", mean(early)),
                 sprintf("Late  nodes (q4)\nmean k=%.1f", mean(late))),
     col     = c("#C44E52", "#4C72B0"), lwd = 2, bty = "n", cex = 0.85)

## (C) Mean-field theory:  $k_i(t) = m * \sqrt{t/i}$ 
t_max <- n_ba_fm
t_range <- seq(m_ba, t_max)

```

```

plot(NULL, xlim = c(0, t_max), ylim = c(0, m_ba * sqrt(t_max / 1) * 1.05),
     xlab = "Time t",
     ylab = expression(paste("Expected degree ", k[i](t))),
     main = expression(paste("Mean-Field: ", k[i](t) == m * sqrt(t/i))))

born_at <- c(1, 10, 100, 500)
mf_cols <- c("#C44E52", "#DD8452", "#4C72B0", "#aec7e8")
for (j in seq_along(born_at)) {
  i_born <- born_at[j]
  t_plot <- t_range[t_range >= i_born]
  k_th <- m_ba * sqrt(t_plot / i_born)
  lines(t_plot, k_th, col = mf_cols[j], lwd = 2)
}
legend("topleft",
      legend = paste0("node born at t=", born_at),
      col = mf_cols, lwd = 2, bty = "n", cex = 0.85)

```

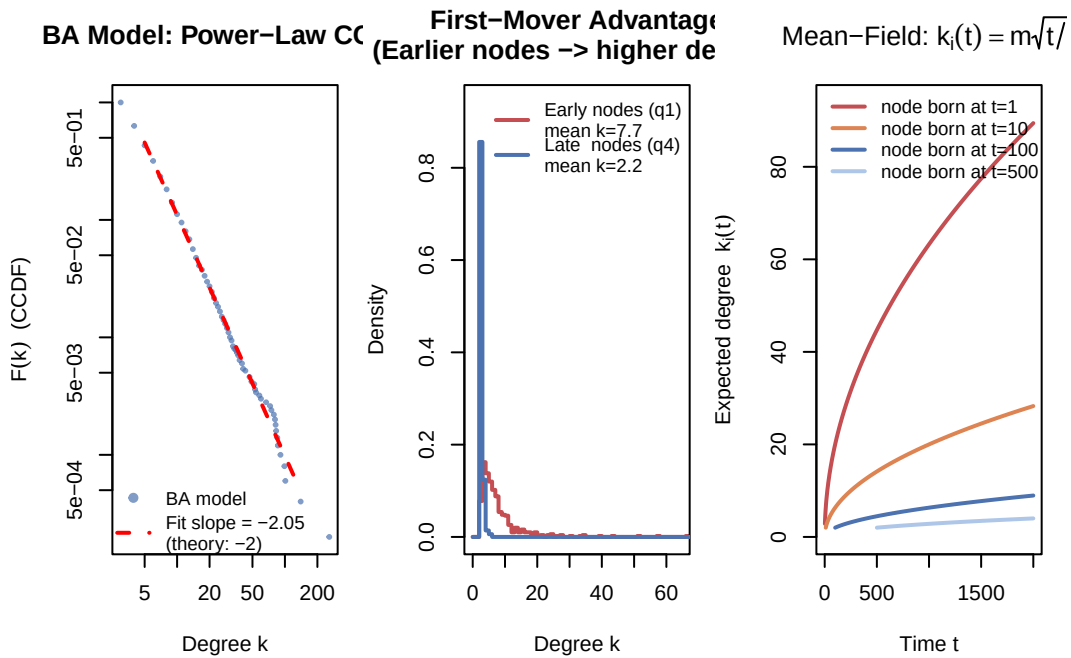


Figure 5: Barabási–Albert model: power-law degree distribution with $m=3$, plus first-mover advantage.

```

cat(sprintf("BA model (n=5000, m=3): <k>=%.2f (theory 2m=%d)\n",
           mean(degs_ba), 2 * 3))

```

BA model (n=5000, m=3): $\langle k \rangle = 6.00$ (theory $2m=6$)

```
cat(sprintf("Estimated CCDF slope: %.3f (theory: -2, so  = 3)\n", slope_fit))
```

Estimated CCDF slope: -2.052 (theory: -2, so = 3)

```
cat(sprintf("\nFirst-mover advantage:\n"))
```

First-mover advantage:

```
cat(sprintf("  Early nodes (born q1): mean degree = %.2f\n", mean(early)))
```

Early nodes (born q1): mean degree = 7.66

```
cat(sprintf("  Late  nodes (born q4): mean degree = %.2f\n", mean(late)))
```

Late nodes (born q4): mean degree = 2.17

```
cat(sprintf("  Ratio: %.1fx\n", mean(early) / mean(late)))
```

Ratio: 3.5x

5.3 The Jackson–Rogers Hybrid Model

The BA model generates power-law degree distributions but fails on:

- **Clustering:** $C \rightarrow 0$ as in ER (pure random attachment produces no triangles)
- **Degree assortativity:** BA has neutral assortativity (no correlation between a node's degree and its neighbours' degrees)

Jackson & Rogers (2007, AER) propose a two-stage attachment rule:

1. Each new node randomly meets m_r existing nodes (*uniform* attachment)
2. Then explores m_n additional nodes by following the *neighbours* of the first m_r nodes (network search)

This generates both a (roughly) power-law degree distribution AND positive clustering (because node i connects to both node j and j 's neighbours, creating triangles). The ratio $r = m_r/m_n$ controls the trade-off:

- $r \rightarrow \infty$ (all uniform): exponential distribution, zero clustering
 - $r \rightarrow 0$ (all network search): power law, positive clustering
 - Intermediate r : interpolates; fits many real networks well
-

6 Stochastic Block Models: Networks with Community Structure

6.1 From Descriptive to Generative

We have seen community detection as an *algorithmic* problem (Lecture 3). But we can also approach it from a modelling perspective: postulate that the network was *generated* by a block model, and infer the parameters.

This is the **Stochastic Block Model (SBM)**, which generalises ER to allow heterogeneous linking probabilities between groups.

6.2 Model Definition

Assign each node i to a group $z_i \in \{1, \dots, k\}$. Given the group assignments, edges are drawn independently:

$$A_{ij} \sim \text{Bernoulli}(M_{z_i, z_j})$$

where M is a $k \times k$ symmetric **mixing matrix** of connection probabilities.

Special cases: - $M_{uv} = p$ for all u, v : recovers Erdős–Rényi - $M_{uu} = p_{\text{in}}$, $M_{uv} = p_{\text{out}}$ for $u \neq v$: the **planted partition model**; the simplest model with community structure

6.3 The Planted Partition Model

Set $p = (2c + \varepsilon)/(2n)$ and $q = (2c - \varepsilon)/(2n)$. Here c is the average degree, and ε controls community *sharpness*:

- $\varepsilon = 0$: ER (no community signal)
- $\varepsilon \gg 0$: very distinct communities

Detectability threshold (Decelle et al. 2011): below the threshold $\varepsilon > 2\sqrt{c}$, no algorithm can reliably detect communities; not because it's computationally hard, but because the signal is *statistically indistinguishable* from noise. This is a phase transition in the informativeness of the data.

6.4 MLE Inference for SBM

Given a network G and assuming k communities, find the group assignments z and mixing matrix M that maximise the likelihood. Since each pair (i, j) generates a Bernoulli with parameter M_{z_i, z_j} :

$$\ln \mathcal{L}(G \mid z, M) = \sum_{i < j} \left[A_{ij} \ln M_{z_i z_j} + (1 - A_{ij}) \ln(1 - M_{z_i z_j}) \right]$$

Maximising over M for fixed z : standard Binomial MLE gives

$$\hat{M}_{uv} = \frac{E_{uv}}{N_{uv}}$$

where E_{uv} = observed edges between groups u and v , and N_{uv} = total possible edges between them.

Maximising over z is NP-hard in general, but greedy heuristics work well.

```
# NMI helper using base R (no sklearn dependency)
nmi_r <- function(a, b) {
  # mutual information via contingency table
  tab <- table(a, b)
  n_obs <- sum(tab)
  p_xy <- tab / n_obs
  p_x <- rowSums(p_xy)
  p_y <- colSums(p_xy)
  # H(X), H(Y), H(X,Y)
  H_x <- -sum(p_x[p_x > 0] * log(p_x[p_x > 0]))
  H_y <- -sum(p_y[p_y > 0] * log(p_y[p_y > 0]))
  H_xy <- -sum(p_xy[p_xy > 0] * log(p_xy[p_xy > 0]))
  MI <- H_x + H_y - H_xy
  if ((H_x + H_y) == 0) return(0)
  2 * MI / (H_x + H_y)
}

set.seed(42)
n_sbm <- 200
k_sbm <- 2
c_sbm <- 5.0
thresh <- 2 * sqrt(c_sbm)
eps_vec <- c(0.0, thresh * 0.8, thresh * 2.0)
```

```

par(mfrow = c(1, 3), mar = c(1, 1, 4, 1))
true_labels <- c(rep(0L, n_sbm / 2), rep(1L, n_sbm / 2))

for (eps in eps_vec) {
  p_in <- (2 * c_sbm + eps) / (2 * n_sbm)
  p_out <- max(0, (2 * c_sbm - eps) / (2 * n_sbm))

  pref_mat <- matrix(c(p_in, p_out, p_out, p_in), nrow = 2)
  g_sbm <- sample_sbm(n_sbm, pref = pref_mat,
                     block.sizes = rep(n_sbm / k_sbm, k_sbm))

  # Community detection via Louvain
  mem <- membership(cluster_louvain(g_sbm))
  nmi_val <- nmi_r(true_labels, mem)

  v_colors <- ifelse(true_labels == 0, "#4C72B0", "#C44E52")
  set.seed(1)
  lay <- layout_with_kk(g_sbm)
  plot(g_sbm,
       layout = lay,
       vertex.color = v_colors,
       vertex.size = 5,
       vertex.label = NA,
       edge.color = "#cccccc",
       edge.width = 0.5,
       main = sprintf("=%.1f (threshold=%.1f)\nNMI=%.3f",
                      eps, thresh, nmi_val))
}
mtext("Planted Partition Model: Detectability\n(Blue/Red = true communities)",
      outer = TRUE, line = -2, font = 2, cex = 1)

```

Planted Partition Model: Detectability

$\epsilon=0.0$ (threshold=4.5) $\epsilon=0.1$ (threshold=4.5) $\epsilon=0.2$ (threshold=4.5) $\epsilon=0.3$ (threshold=4.5) $\epsilon=0.4$ (threshold=4.5) $\epsilon=0.5$ (threshold=4.5) $\epsilon=0.6$ (threshold=4.5) $\epsilon=0.7$ (threshold=4.5) $\epsilon=0.8$ (threshold=4.5) $\epsilon=0.9$ (threshold=4.5)

NMI=0.010 NMI=0.060 NMI=0.468

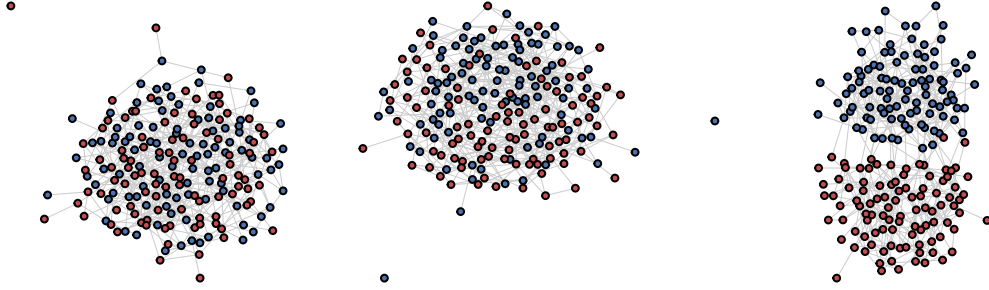


Figure 6: Planted partition model: varying ϵ around the detectability threshold.

6.5 Degree-Corrected SBM (DC-SBM)

A known failure of the basic SBM: when the degree distribution within communities is heterogeneous, the MLE tends to group *high-degree* nodes together rather than finding true communities.

The **DC-SBM** (Karrer & Newman 2011) adds per-node *propensity parameters* $\gamma_i \geq 0$:

$$A_{ij} \sim \text{Poisson}(\gamma_i \gamma_j M_{z_i z_j})$$

The γ_i absorbs degree heterogeneity, allowing the block structure M to capture community membership without being confounded by degree. MLE gives $\hat{\gamma}_i = k_i / \kappa_{z_i}$ (node i 's degree as a fraction of its group's total degree) and $\hat{M}_{uv}$ as before.

7 Summary and What's Next

7.1 Model Comparison

	Power-law degree	High clustering	Short paths	Communities
Erdős-Rényi	No (Poisson)	No (C→0)	Yes	No
Configuration	Arbitrary	No (C→0)	Yes	No

Watts-Strogatz	No (Poisson-like)	Yes	Yes	No
Barabási-Albert	Yes (=3)	No (C=0)	Yes (hubs)	No
Jackson-Rogers	Yes (tunable)	Yes	Yes	No
SBM	No (Poisson)	Within-block	Yes	Yes
	Tractable?			
Erdős-Rényi	Fully			
Configuration	Mostly			
Watts-Strogatz	Numerical			
Barabási-Albert	Mean-field			
Jackson-Rogers	Numerical			
SBM	MLE			

7.2 What We Cannot Yet Answer

We have good generative models, but three big inferential questions remain:

1. **Given a real network, which model generated it?** Model selection requires likelihood comparison (BIC/AIC) or Bayesian approaches.
2. **How do we find communities in an observed network?** SBM gives us the generative story, but inference (finding z from G) is hard computationally. This is the community detection problem; the focus of the next lecture.
3. **How do we predict missing edges or node labels?** Even with a good model, predicting new edges (link prediction) or unseen node attributes (semi-supervised classification) requires additional ML machinery.

These three questions; community detection, link prediction, node classification; are the subject of **Lecture 3: Machine Learning with Network Data**.

7.3 References

- Erdős, P. & Rényi, A. (1960). On the evolution of random graphs. *Publ. Math. Inst. Hung. Acad. Sci.*, 5, 17–61.
- Newman, M. E. J. (2013). *Networks: An Introduction*. Ch. 11–15.
- Watts, D. J. & Strogatz, S. H. (1998). Collective dynamics of ‘small-world’ networks. *Nature*, 393, 440–442.
- Barabási, A.-L. & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286, 509–512.
- Jackson, M. O. & Rogers, B. W. (2007). Meeting strangers and friends of friends. *American Economic Review*, 97(3), 890–915.

- Jackson, M. O. (2019). The friendship paradox and systematic biases. *Journal of Political Economy*, 127(5), 2219–2252.
- Karrer, B. & Newman, M. E. J. (2011). Stochastic blockmodels and community structure in networks. *Physical Review E*, 83, 016107.
- Decelle, A. et al. (2011). Asymptotic analysis of the stochastic block model. *Physical Review E*, 84, 066106.
- Feld, S. L. (1991). Why your friends have more friends than you do. *American Journal of Sociology*, 96(6), 1464–1477.